

Interim Submission Summary: AI-Powered Timeline Travel Planning and Assistance Platform

Team_Egress

June 20, 2026

Contents

1	Problem Statement	3
1.1	Key Issues Identified	3
2	Chosen Domain	4
2.1	Domain Focus Areas	4
3	Solution Outline	4
3.1	Core Features	5
4	System Architecture	5
4.1	User-Facing Plane	6
4.2	Administrative Plane	6
4.3	Agentic Backend	7
4.4	Shared Memory and State	7
4.5	Domain Services	7
4.6	External APIs and Feeds	8
4.7	Proactive Alerts Pipeline	8
4.8	Key Architectural Properties	8
4.9	Expected User Experience	9

5	How We Plan to Apply AI	9
5.1	AI Capabilities Used	9
5.2	Planned AI Workflow	9
6	Scope, Feasibility, and Evaluation	10
6.1	Scope for This Phase	10
6.2	Feasibility and Risks	10
6.3	Evaluation Plan	10
7	Highlighted Key Points	11

1 Problem Statement

Modern travel planning is fragmented, manual, and difficult to adapt once a trip begins. Travelers move between many disconnected platforms to compare attractions, transport, accommodation, food, weather, local events, cultural expectations, and emergency information. This makes planning slow beforehand and stressful during the journey, especially when conditions change unexpectedly.

The difficulty is sharper in Sri Lanka, where a compact geography packs in beaches, heritage sites, temples, wildlife parks, hill-country routes, food experiences, festivals, adventure activities, and local markets. The same density that makes the country attractive also makes it hard to plan: a single trip may require long-distance transfers, local transport, hotel check-ins, activity reservations, temple etiquette, weather-sensitive outdoor activities, and route delays to all line up within a limited schedule.

Existing tools mostly produce *static* itineraries. They suggest places but do not model the trip as a timeline with dependencies between activities. When a train is delayed, a road is blocked, a storm affects a hike, or a venue closes early, the traveler is left to manually rework the rest of the day, often resulting in missed bookings, rushed travel, invalid check-in times, unsafe routes, and wasted time.

Two further gaps stand out. First, recommendations are typically generic and reactive: they do not continuously reflect a traveler’s budget, style, dietary needs, accessibility requirements, preferred transport, and interests, nor the current state of the trip. Second, cultural awareness is often missing. Many Sri Lankan destinations are temples, religious sites, festivals, and culturally sensitive communities, where travelers may not know the dress codes, photography restrictions, behavioral expectations, or religious timing constraints, and can unintentionally cause offense or miss important context.

The core problem is therefore the absence of an intelligent, adaptive, context-aware travel assistant that represents a trip as a *living timeline*, understands each item in detail, monitors changing conditions, and helps travelers make better decisions before and during the journey.

1.1 Key Issues Identified

- Travel information is scattered across platforms and service providers, forcing manual coordination of transport, stays, activities, meals, prices, and local guidance.
- Current itinerary tools are static; delays and disruptions are not automatically propagated across the rest of the plan.
- Recommendations are generic and do not reflect user preferences or real-time context.
- Travelers often lack cultural guidance for temples, festivals, religious locations, and local customs.
- There is no unified timeline that combines transport, accommodation, activities, food, emergency support, risks, and constraints in one place.

2 Chosen Domain

The chosen domain is **Tourism and Travel Planning**, with a specific focus on Sri Lanka. The project combines travel technology, intelligent scheduling, constraint-aware planning, recommendation systems, and agentic AI.

The platform does not merely recommend attractions; it manages the complete journey as a sequence of timeline nodes. Each node represents a concrete part of the trip, such as a hotel stay, a transport segment, an attraction visit, a meal stop, a temple visit, or a weather-sensitive outdoor activity. Modeling travel this way lets the platform reason about how one activity affects another and respond intelligently when plans change.

Sri Lanka is a suitable focus because travel there involves many interacting real-world variables, including coastal travel, mountain routes, wildlife parks, ancient cities, religious locations, train journeys, tuk-tuk transfers, and weather-dependent activities. A timeline-based system can help travelers make better use of this diversity while reducing uncertainty and stress.

The domain also emphasizes *in-trip* support. Most platforms focus on pre-trip planning, but travelers also need help while the trip is happening: what to do after a delay, whether an attraction is still reachable, which cultural rules apply at the next temple, where to eat nearby, or whether a route is safe during rain.

2.1 Domain Focus Areas

- Timeline-based itinerary planning and management.
- Real-time trip assistance before and during travel.
- Dynamic disruption handling and delay propagation.
- Personalized recommendations for activities, routes, dining, stays, and time slots.
- Cultural guidance for religious, social, and local contexts.
- Context-aware interaction in which users do not need to repeat trip details.

3 Solution Outline

The proposed solution is an **AI-powered Timeline Travel Planning and Assistance Platform**. The platform represents a trip as a time-ordered timeline of actionable nodes. Each node carries the information needed to understand, preview, modify, and monitor that part of the journey, including transport details, stay information, the visiting location, food and dining options, emergency contacts, cultural context, constraints, and risk indicators. This turns an itinerary from a list of places into a structured plan that can be monitored, edited, and adapted as the trip progresses.

At the center is the **Trip Orchestrator Agent**, which receives user requests, establishes the cur-

rent trip context, identifies the relevant timeline node, and coordinates **four specialized agents** (Planner, Logistics, Disruption, and Culture & Etiquette). The orchestrator combines their outputs, resolves conflicts, and updates the timeline.

For example, if a traveler reports traffic on the way to Kandy, the orchestrator identifies the active node, the Disruption Agent estimates the delay and updates the node, and the system checks the remaining timeline. If the delay threatens a hotel check-in, a dinner booking, a temple visit, or a train connection, the platform flags the conflict and proposes recovery actions: push later nodes, drop a low-priority activity, take a faster route, replace an activity, or leave the conflict flagged for review.

Node states make the timeline easy to read at a glance. A *green* node has no known problems; a *red* node has a disruption, conflict, or risk that needs attention; a *purple* node has had its tasks completed. The currently active node and upcoming nodes are highlighted distinctly so the traveler always knows where they are in the day.

Users can perform full timeline operations: create, update, reorder, reschedule, or delete nodes, and open any node to preview route options, cultural notes, nearby dining, emergency services, and risk indicators. Travelers interact through a web and mobile app and a WhatsApp/Telegram bot, so they are not tied to one interface, while partners and platform maintainers use a separate admin/partner console.

3.1 Core Features

- **Timeline itinerary:** converts a trip into a sequence of actionable, time-aware nodes.
- **Node previews:** transport, stay, visit, food, emergency, cultural, and risk details per item.
- **Disruption handling:** users report delays, closures, storms, and traffic, and the system also detects disruptions automatically from external alerts.
- **Delay propagation:** automatically checks how one delay affects future reservations, activities, routes, meals, and check-ins.
- **Personalized recommendations:** suggests better routes, attractions, food, stays, time slots, and replacement activities.
- **Cultural guidance:** dress codes, etiquette, photography rules, religious expectations, and festival impacts.
- **Implicit context:** infers the active trip and node from the session, schedule, location, and timeline state, so users need not repeat trip details.

4 System Architecture

The platform follows a layered architecture with a clear separation between the user-facing request path, an administrative plane, the agentic backend, a shared memory layer, the domain services, and the external feeds. End-user requests enter through an **API Gateway / Backend Router**

and reach the Trip Orchestrator, which delegates to the specialized agents; the agents draw on domain services, and only those services talk to external APIs. This strict layering keeps each concern in one place: an agent never calls an external API directly.

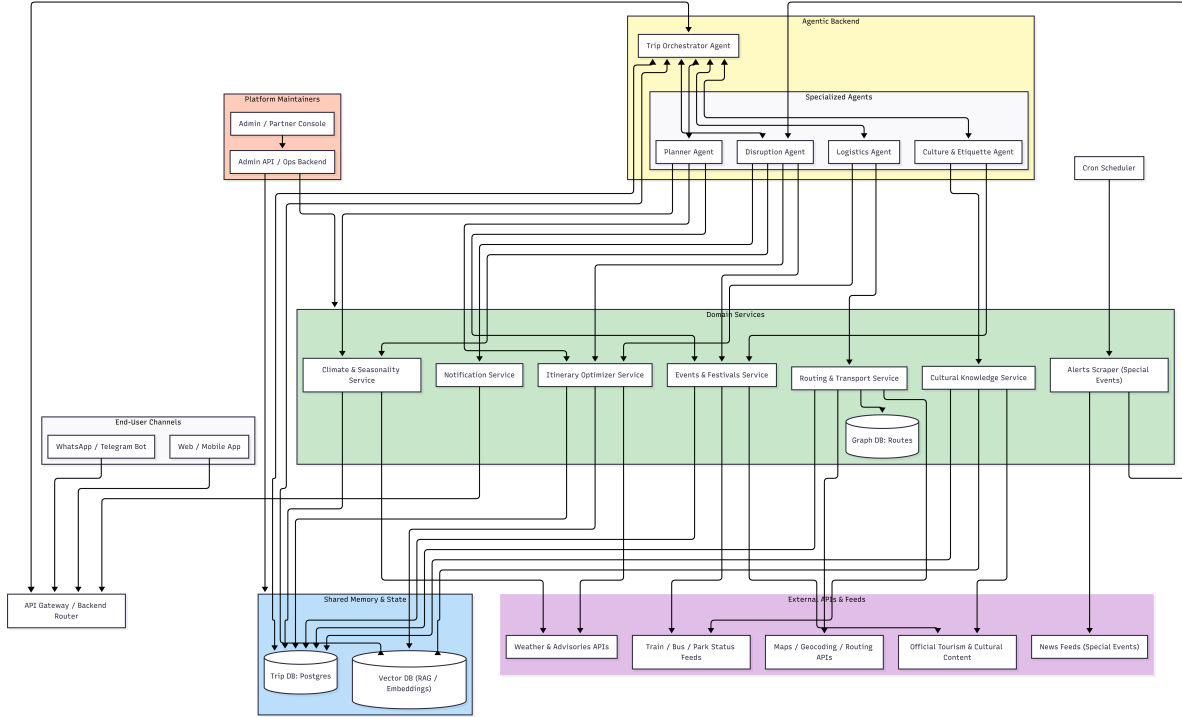


Figure 1: High-level architecture of the AI-powered timeline travel planning platform.

4.1 User-Facing Plane

Two end-user channels front the system: a **Web / Mobile App** and a **WhatsApp / Telegram bot**. Both communicate through the **API Gateway / Backend Router**, which exchanges messages bidirectionally with the Trip Orchestrator and also serves as the egress path for outbound notifications back to users.

4.2 Administrative Plane

Platform maintainers and partners use a separate **Admin / Partner Console** that connects through an **Admin API / Ops Backend**. This plane is independent of the traveler request path and is used to operate and configure the domain services and to inspect and manage the shared memory and state.

4.3 Agentic Backend

This layer contains the **Trip Orchestrator Agent** and the **four specialized agents**, which communicate bidirectionally with the orchestrator.

- **Trip Orchestrator Agent** — understands the request, identifies the active trip and node, gathers context, assigns work to the agents, resolves conflicts between their outputs, and writes the final timeline. It reads from and writes to both shared memory stores.
- **Planner Agent** — builds and optimizes the itinerary using the Itinerary Optimizer, Climate & Seasonality, and Events & Festivals services. Personalized recommendations are produced here, drawing on the Vector DB for preference and similarity matching.
- **Logistics Agent** — evaluates transport options, route feasibility, durations, and transfer dependencies using the Routing & Transport and Itinerary Optimizer services.
- **Disruption Agent** — handles delays, closures, cancellations, and storms by recomputing affected nodes through the Itinerary Optimizer, Climate & Seasonality, and Events & Festivals services, and proposing recovery actions. It also receives proactive signals from the Alerts Scraper and triggers the Notification Service.
- **Culture & Etiquette Agent** — supplies temple etiquette, clothing requirements, photography rules, and festival impacts via the Cultural Knowledge and Events & Festivals services.

4.4 Shared Memory and State

A separate, shared layer holds the system’s persistent state: a **Trip DB (PostgreSQL)** that stores the canonical trip, bookings, timeline, and node states, and a **Vector DB (RAG / embeddings)** that stores preferences, past decisions, similar trips, and contextual knowledge for retrieval. The Vector DB is what enables implicit context and increasingly relevant personalization over time. Both the Trip Orchestrator and the administrative plane access this layer.

4.5 Domain Services

Seven domain services, backed by a route **Graph DB**, sit between the agents and the outside world:

- **Itinerary Optimizer Service** — orders and re-orders activities under time, location, and opening-hour constraints (uses Weather APIs, Trip DB, Vector DB).
- **Routing & Transport Service** — computes routes and transfers (uses the route Graph DB, Maps APIs, transport feeds, and Trip DB).
- **Climate & Seasonality Service** — assesses weather and seasonal risk (uses Weather APIs and Trip DB).
- **Events & Festivals Service** — tracks events and festival impacts (uses transport feeds, official tourism content, and Trip DB).

- **Cultural Knowledge Service** — holds cultural and etiquette knowledge (uses official tourism content, Trip DB, and Vector DB).
- **Alerts Scraper (Special Events)** — triggered periodically by a Cron Scheduler; reads external news feeds for special events that could disrupt travel and signals the Disruption Agent when one is found.
- **Notification Service** — triggered by the Disruption Agent; pushes updates to users through the API Gateway.
- **Graph DB (Routes)** — stores the route and transport network used for routing and reachability.

4.6 External APIs and Feeds

Five external sources are consumed only through the domain services: **Weather & Advisories APIs**, **Maps / Geocoding / Routing APIs**, **Train / Bus / Park Status feeds**, **Official Tourism & Cultural Content**, and **News Feeds (Special Events)**.

4.7 Proactive Alerts Pipeline

A **Cron Scheduler** periodically triggers the Alerts Scraper (Special Events), which scans external news feeds for events that could affect travel, such as processions, road closures, or strikes. When it finds a relevant event, it signals the Disruption Agent, which evaluates the affected timeline and, where needed, triggers the Notification Service to push an update to the user through the API Gateway. This lets the platform surface disruptions proactively, not only when a user reports them.

4.8 Key Architectural Properties

- **Strict layering:** agents call domain services; only domain services call external APIs. This isolates external dependencies and makes individual services replaceable.
- **Single source of truth:** the shared Trip DB holds the canonical timeline and node states, read and written consistently across layers.
- **Separate administrative plane:** maintainers operate the domain services and inspect or manage the shared state without going through the traveler request path.
- **Proactive disruption pipeline:** Cron Scheduler → Alerts Scraper → Disruption Agent → Notification Service → API Gateway → user, complementing user-reported issues.
- **Personalization layer:** the Vector DB (RAG / embeddings) underpins implicit context and learning across the orchestrator, Itinerary Optimizer, and Cultural Knowledge service.

4.9 Expected User Experience

The traveler shares goals, dates, preferences, budget, and constraints, and the platform generates a timeline of activities, stays, routes, meals, and contextual notes. During the trip, the traveler opens the timeline, inspects nodes, asks questions, and reports issues. When something changes, the platform does not give generic advice: it updates the timeline, checks downstream consequences, highlights conflicts, and explains recovery options, acting as both a planning assistant and a real-time travel companion.

5 How We Plan to Apply AI

AI is applied through an agentic architecture in which the Trip Orchestrator coordinates four specialized agents, each contributing a distinct kind of intelligence, with the orchestrator combining their outputs into safe, practical, personalized decisions. The agent responsibilities are detailed in the architecture section above; this section summarizes the AI techniques used and the end-to-end workflow.

5.1 AI Capabilities Used

- Natural language understanding to interpret requests and disruption reports.
- Multi-agent reasoning to divide travel tasks across the Planner, Logistics, Disruption, and Culture & Etiquette agents.
- Constraint-aware planning over time, location, opening hours, bookings, and transport limits, delegated to the Itinerary Optimizer.
- Real-time adaptation to recompute the timeline when conditions change.
- Recommendation generation based on preferences, goals, context, and availability, using the Vector DB.
- Semantic memory to learn behavior and support implicit context.
- Retrieval from domain services for weather, routes, transport, events, dining, and cultural information.
- Conflict resolution that balances preferences, hard constraints, safety, cost, and convenience.
- Explainable responses so users understand why a change or recommendation is proposed.

5.2 Planned AI Workflow

1. The user submits a request, asks a question, or reports a disruption (or the Cron-driven Alerts Scraper proactively signals the Disruption Agent).
2. The Trip Orchestrator identifies the active trip, node, intent, and required context from the Trip DB and Vector DB.

3. The relevant agents call their domain services, which retrieve data from external APIs and feeds.
4. Each agent analyzes the problem from its specialty: planning, logistics, disruption, or culture.
5. The orchestrator compares outputs, resolves conflicts, and prioritizes user goals and hard constraints.
6. The timeline is updated with new times, node states, warnings, recommendations, or alternatives.
7. The response is delivered in a clear, actionable format.
8. User decisions and feedback are stored to improve future personalization.

6 Scope, Feasibility, and Evaluation

6.1 Scope for This Phase

The interim prototype targets the end-to-end timeline experience: generating a Sri Lanka itinerary, previewing nodes, and handling at least one full disruption-and-recovery cycle with delay propagation across the remaining timeline. The Trip Orchestrator and all four specialized agents are in scope, as are the Itinerary Optimizer, Routing & Transport, Climate & Seasonality, and Cultural Knowledge services.

6.2 Feasibility and Risks

- **Data availability:** real-time Train / Bus / Park status for Sri Lanka may have limited or unofficial public APIs. Where live feeds are unavailable, the team will fall back to scheduled data, scraped advisories, and user-reported status, and document the limitation.
- **Routing accuracy:** hill-country and rural routes can be poorly represented in mapping APIs; the route Graph DB lets us encode local corrections.
- **Cultural content accuracy:** etiquette and festival guidance will be sourced from official tourism content and reviewed, since errors here carry real social cost.

6.3 Evaluation Plan

- **Functional:** verify that a reported delay correctly propagates and that conflicts (check-ins, bookings, connections) are detected and surfaced.
- **Quality:** rate the relevance of recommendations and the correctness of cultural guidance against a small benchmark of Sri Lanka trips.
- **Usability:** measure whether implicit context reduces the need for users to restate trip details, and gather qualitative feedback on the timeline view.

7 Highlighted Key Points

- The core idea is a dynamic travel timeline, not a static itinerary list, with each node carrying operational, cultural, contextual, emergency, dining, and risk information.
- A Trip Orchestrator coordinates exactly four specialized agents (Planner, Logistics, Disruption, Culture & Etiquette) over a strictly layered architecture of seven domain services and five external feeds.
- Delay propagation is central: one disrupted activity is checked against the rest of the trip, and disruptions can be detected proactively via a Cron-driven pipeline (Alerts Scraper → Disruption Agent → Notification Service → user).
- A shared memory layer (Trip DB plus a RAG Vector DB) supports implicit context and learning, so the system improves over time and users need not repeat trip details.
- The focus on Sri Lanka grounds the work in real constraints: weather, geography, transport delays, religious sites, and cultural expectations.
- The goal is to reduce planning effort, decision fatigue, and travel stress while improving safety, personalization, and cultural awareness.